

PROJECT SOUNDING LINE

The Software Verification and Parallel Test Infrastructure System

PART II

Execution Infrastructure and Parallel Runtime

Version 1.2 Amendment Edition

Amendment purpose

Make the Sounding Line runtime conform to its own scheduling and isolation architecture. Version 1.2 mandates demand-driven resource preparation, immutable prepared environments, certified database baselines, worker bundling with per-suite receipts, typed resource capacities, multi-lane browser execution, dynamic ports, dependency-aware scheduling, failure-aware cancellation, and critical-path telemetry so authoritative proof becomes dramatically faster without becoming weaker.

Effective August 11, 2026 upon protected-mainline acceptance | Document ID CS-SL-P2-001 v1.2

A. Document Control and Amendment Authority

Field	Value
Program	Project Sounding Line
Subsystem	The Software Verification and Parallel Test Infrastructure System
Document	Governing Architecture and Product Requirements Document - Part II, Version 1.2 Amendment Edition
Base authority	Version 1.0, July 28, 2026, plus the accepted Version 1.1 Amendment Edition dated August 10, 2026.
Version	1.2
Effective date	August 11, 2026 upon acceptance through the protected Sounding Line mainline path.
Document ID	CS-SL-P2-001 v1.2
Status	Governing amendment candidate; becomes authoritative on protected-mainline acceptance. Additive and superseding only where clauses are explicitly named.
Amendment trigger	Observed growth in authoritative-run wall time and repeated setup/revalidation during the August 2026 concurrent project wave, including suite-per-runner setup repetition, broad exclusive serialization, browser provisioning duplication, and late expensive failure discovery.
Primary objective	Maximize verification throughput without reducing coverage, evidence integrity, source binding, risk floors, or release confidence.

Normative incorporation rule

Version 1.0 and Version 1.1 remain in force except where this Version 1.2 amendment explicitly adds, clarifies, or supersedes a clause. Version 1.2 is not permission to run fewer required tests or accept weaker evidence. It requires Sounding Line to obtain the same or stronger decision confidence with the least redundant execution that can be proven correct. Where Version 1.1 defines queueing and multi-lane target behavior, Version 1.2 makes the hosted and local runtime responsible for concrete resource-aware preparation, bundling, capacity profiles, reusable immutable setup, and conformance telemetry.

The three-document authority split remains unchanged: Part I defines verification meaning, Part II defines execution machinery, and Part III defines repository, Codex, CI, mainline, and release enforcement.

1. Amendment Summary and Supersession Map

Existing area	Version 1.2 addition
3 Execution Graph	Adds worker-bundle nodes and explicit prepared-artifact dependencies while preserving logical suite nodes as evidence boundaries.
4 Scheduler Behavior	Replaces broad parallel/exclusive grouping with typed resource capacities and dependency-ready scheduling; global serialization requires a real shared resource.
5 Runtime Lease Broker	Adds bundle leases, dynamic endpoint allocation, certified baseline references, and lane-specific capacity accounting.

Existing area	Version 1.2 addition
6 Resource Catalog	Adds setup classes, prepared artifact identities, browser-engine caches, dependency environments, and baseline resources.
8-13 Baselines/DB/fixtures	Defines certified SQLite baseline build-once/clone-many behavior and integrity contracts.
16-18 Browser/ports/sharding	Makes two isolated hosted browser lanes the normal minimum topology; removes fixed-port global mutex; installs only declared engines.
25 Timeout Architecture	Adds bootstrap/setup and queue/runner timing to critical-path evidence; no test timeout may absorb infrastructure setup delay.
30-33 Capacity/local/CI	Defines resource profiles, hosted runner bundle sizing, queue-exposure minimization, and prepared environment reuse.
39 Acceptance Matrix	Adds warm/cold throughput, worker-bundling, baseline cloning, dual-browser-lane, failure-cancellation, and conformance scenarios.

2. Target Authoritative Execution Topology

The authoritative runtime SHALL distinguish logical evidence nodes from physical workers. Logical suites remain independently registered and evidenced; physical workers are an optimization surface that may execute one or more compatible suites.

```

SEALED PLAN
|
+-- source/invalidation guard
|
+-- PREPARE / RESTORE IMMUTABLE ARTIFACTS
|   dependency environment
|   generated client identity
|   certified SQLite baseline
|   browser engine cache
|
+-- WORKER BUNDLES
|   node-worker-a -> unit/component/static receipts
|   node-worker-b -> unit/component/static receipts
|   browser-lane-a -> browser receipts
|   browser-lane-b -> browser receipts
|   build-lane     -> build receipt
|   database-lane  -> database/provider receipts
|
+-- FINALIZER
    fresh + preserved + rebound receipts
    runtime-conformance receipt
    cleanup proof
    RELEASE_GO / RELEASE_NO

```

Evidence boundary rule

A GitHub Actions job, container, process, or machine is not a verification evidence boundary. The registered Sounding Line suite/obligation is. Bundling may reduce setup repetition, but it may not merge receipt identity, hide case counts, or make one suite's cleanup/result ambiguous.

3. Demand-Driven Worker Preparation

The governed worker SHALL read the sealed plan node/bundle before expensive setup. Preparation is derived from declared resources and adapter needs. The generic worker template may not unconditionally provision database, browsers, build output, or services.

Worker class	Typical resources	Required preparation	Forbidden implicit preparation
node-only	node-slot, vitest-worker-pool, scanner	Checkout/restore dependency environment; execute tests/static/privacy scanner as declared.	No Prisma migration/seed; no browsers; no app server.
prisma/schema	prisma-sqlite-client or mysql-schema	Restore/generate exact client; schema validate.	No seeded database unless sqlite-clone/MySQL namespace declared.
database	sqlite-clone / mysql namespace	Clone certified baseline or provision namespace; validate identity.	No browser unless declared.
browser	browser-chromium and/or browser-webkit + app port + sqlite clone	Restore exact browser engine(s); allocate port; clone DB; start run-owned server; readiness identity check.	Never install undeclared browser engine.
build	production-build-directory	Restore dependency environment; produce source-bound build output.	No browser/database by default.

3.1 Resource-plan preflight

1. Load sealed plan and exact bundle assignment.
2. Verify current policy/resource catalog digest matches sealed plan requirements.
3. Compute preparation actions from resource declarations; record them before execution.
4. Reject preparation action that has no declared/transitive resource requirement.
5. Emit setup receipt including restored/cache identities and cold/warm status.

4. Prepared Immutable Environment Artifacts

Repeated reconstruction of identical immutable setup SHALL be replaced by source-qualified or platform-qualified prepared artifacts. Reuse is allowed only when complete identity matches and the mutable boundary remains run-owned.

Artifact class	Identity inputs	Reuse model	Invalidation
Dependency Environment	OS/architecture; Node version; package-lock digest; package manager version; approved install-script policy; dependency artifact digest.	Read-only or clean-copy restore into worker.	Never reuse if lock/toolchain/platform identity differs.
Generated Client Artifact	Prisma version; schema digest; generator config; platform; generated artifact digest.	Restore exact generated output or regenerate once when identity misses.	Never assume a client generated for a different schema is valid.
Browser Environment	OS; Playwright version; browser name/build; browser cache digest.	Restore exact required engine(s).	No “all browsers” install when one engine is required.
Certified SQLite Baseline	SQLite/Prisma versions; schema+migration digest; seed builder digest; fixture policy; baseline DB SHA-256.	Copy into run-owned DB clone.	Baseline itself is immutable; every mutating run uses a copy.

4.1 Prepared artifact trust chain

- Artifact creation must itself be governed and produce a preparation receipt.
- Restored artifact digest must match the manifest before any suite uses it.

- Mutable tools may copy from the artifact but may not mutate the canonical prepared artifact.
- Cache miss is not a test failure. It triggers legitimate preparation and updates timing telemetry.
- Cache hit is not correctness evidence by itself; correctness comes from exact identity verification plus the suite result.

5. Certified SQLite Baseline: Build Once, Clone Many

The current migration chain may contain dozens of migrations. Applying and seeding that chain separately for every unit/component/browser worker is prohibited unless a specific suite is explicitly testing migration-from-zero behavior. Ordinary suites SHALL consume a certified baseline or an equivalent immutable prepared state.

```

BASELINE KEY
= digest(
  sqlite/prisma version,
  schema.sqlite.prisma,
  all migration contents,
  seed builder + fixture policy,
  generated client contract
)

cache miss -> build empty DB -> apply migrations -> seed -> integrity checks -> SHA-256 ->
baseline manifest
cache hit -> verify manifest + SHA-256

per run: certified-baseline.db -> COPY -> run-owned-<runId>.db

```

5.1 Baseline acceptance checks

- All migrations applied in deterministic order with no pending migration.
- Seed preset is synthetic, privacy-safe, and versioned.
- Foreign-key/integrity checks pass.
- Baseline SHA-256 and manifest are recorded.
- Baseline is marked immutable; post-run digest is checked if a worker mounts it read-only or as a copy source.
- Suites that explicitly test migration behavior declare a migration resource and may bypass the prepared baseline for that obligation only.

6. Worker Bundling and Per-Suite Receipt Integrity

Tiny suites SHALL not automatically receive one fresh hosted VM each. The scheduler may assign multiple compatible suites to one worker using duration/resource-aware bin packing. The worker executes suites independently and emits one receipt per suite.

6.1 Bundle eligibility

- Suites have compatible environment/toolchain identity.
- Suites do not require mutually exclusive mutable resources.
- Suite hard budgets and cleanup semantics remain enforceable individually.
- Bundling does not obscure case discovery/count reconciliation.
- Failure of one suite does not erase completed receipts from sibling suites; continuation/cancellation follows the graph policy.

6.2 Suggested hosted classes

Class	Work	Normal capacity behavior
node-light	Unit/static/policy/catalog/document suites with no	3-4 parallel workers normal target; duration-

Class	Work	Normal capacity behavior
	database/browser need.	balanced.
node-db	Component suites needing run-owned DB clone but no browser.	1-3 workers according to capacity.
browser	Journey/accessibility/browser suites.	At least 2 isolated hosted lanes when provider capacity permits.
build	Production build and build-sensitive proof.	1 lane; may overlap browser unless actual shared resource exists.
provider	External provider / MySQL / service-specific proof.	Capacity based on provider namespace/rate-limit contract.

Receipt boundary requirement

A bundled worker that executes four suites must produce four independently verifiable suite receipts, each with suite ID, registered/discovered/executed counts where applicable, timing, result, cleanup state, and exact source/policy/plan digests. A bundle-level success string is insufficient.

7. Typed Resource Capacities Replace Global “Exclusive”

A Boolean parallel/exclusive flag is not an adequate model for a multi-project verification runtime. Version 1.2 requires explicit resource capacities and compatibility. A node waits only for resources it actually needs and its hard dependencies.

Resource	Illustrative capacity	Semantics
node-slot	4 or capacity-profile value	Ordinary test execution capacity.
vitest-worker-pool	4 or profile value	CPU/memory budget for unit/component families.
browser-chromium	2 minimum normal hosted target	Independent Chromium browser lanes.
browser-webkit	1 minimum normal hosted target	WebKit lane when required.
production-build-directory	1 per worker/source-bound output	Build output isolation; separate hosted runners do not share this resource.
sqlite-clone	N, bounded by disk/memory	Copies of certified baseline are cheap and run-owned.
external-provider:<id>	Provider-specific	Real shared capacity/rate-limit/namespace; may serialize.
restart-host	1 per actual host	True host-global destructive operation; serialize.

No global mutex without a global resource

Two jobs are not incompatible merely because both are “browser” or “exclusive.” Serialization requires a shared resource identity whose capacity is exhausted. Separate GitHub-hosted runners do not share localhost ports, SQLite files, browser processes, or build directories.

8. Browser Lane Architecture

Browser execution is the most expensive and failure-sensitive resource class. Hosted CI SHALL treat two isolated lanes as the normal minimum topology when provider runner availability permits. Local execution may reduce capacity under interactive pressure but must report the reduction.

Lane element	Contract
Application port	Dynamically leased run-owned port; 3100 may remain a local preference only.
API/helper ports	Run-owned dynamic leases, identity checked.
Database	Run-owned clone/namespace with baseline and source identity.
Browser engine	Only declared engine/build restored; browser context run-owned.
Build output	Source-bound output; may be prepared independently or reused when exact identity permits.
Storage/media root	Run-owned synthetic root.
Evidence/trace root	Run-owned immutable result path after completion.
Readiness proof	Endpoint/receipt must assert run ID, source SHA, database identity, build identity, and port before business traffic.

8.1 Browser provisioning

- Restore Chromium only when `browser-chromium` is declared.
- Restore WebKit only when `browser-webkit` is declared.
- Cache keys include OS/platform and Playwright/browser build identity.
- Browser install/restore time is setup telemetry, never product execution time.
- A browser suite may reuse a prepared application build only when the build identity matches the suite source and configuration requirements.

9. Dependency-Aware Scheduling and Critical Path

Wave numbers may remain an implementation convenience, but they may not create artificial barriers broader than the actual dependency graph. A node becomes eligible when all hard dependencies succeeded and the required resource bundle is available.

```

ELIGIBLE(node) =
  dependencies_passed(node)
  AND outputs_available(node)
  AND resource_bundle_available(node)
  AND source_guard_current(node)

```

9.1 Scheduling priorities

- Critical-path length and decision-unblocking value.
- Scarce-resource opportunity without starving older work.
- Fairness age across projects and gates.
- Historical median/high-percentile execution and setup duration, used only for scheduling.
- Risk class and release criticality.
- Environment pressure and interactive reserve for local execution.

9.2 GitHub Actions translation

GitHub Actions static `needs:` relationships may require fixed worker classes rather than a fully dynamic DAG. The CI implementation SHALL nevertheless preserve semantic dependencies: independent worker classes may start concurrently, and bundle assignment should minimize the number of hosted-runner startups while avoiding large long-tail bundles. The YAML layout is an adapter to the Sounding Line graph, not the architecture itself.

10. Queue Exposure and Runner-Startup Minimization

Hosted runner queue is external infrastructure latency. Sounding Line cannot eliminate it, but it can reduce exposure by requiring fewer physical jobs and by failing stale candidates before expensive runner/resource acquisition whenever possible.

Mechanism	Requirement
Physical worker count	Use bundle workers rather than suite-per-VM micro-sharding when setup dominates execution.
Source guard	Check current candidate/base before expensive browser/provider acquisition and again where gate contract requires.
Prepared artifacts	Restore exact immutable environment rather than reconstructing it per suite.
Short sentinel priority	Allow short decision-unblocking checks to use available capacity without starving older work.
Queue telemetry	Record workflow queue, job queue, resource queue, and active execution separately where platform data permits.

11. Runtime Telemetry and Critical-Path Report

Field	Meaning
githubQueueMs	Workflow/job waiting before runner starts.
checkoutMs	Repository checkout/fetch time.
dependencyRestoreMs	Cache/artifact restore and dependency materialization.
clientGenerateMs	Prisma/generated client preparation.
baselineRestoreMs	Certified DB baseline restore/copy or migration build time.
browserRestoreMs	Playwright browser cache restore/install time by engine.
serverReadyMs	Application/server startup until identity-checked readiness.
suiteExecutionMs	Actual protected assertions/commands.
teardownMs	Cleanup, mutation verification, process/resource release.
artifactUploadMs	Evidence persistence.
finalizerMs	Decision composition and validation.
cacheState	HIT/MISS/REBUILT for each prepared artifact class.

11.1 Completion projection

Every authoritative run SHOULD produce a critical-path summary identifying the longest queue/setup/execution segments, total runner-minutes, useful suite-execution minutes, setup-to-execution ratio, and opportunities for safe reuse. This report is diagnostic evidence; it does not alter the RELEASE_GO semantics.

12. Failure-Aware Cancellation and Evidence Salvage

Fail-fast must follow graph semantics, not a generic matrix switch. Independent completed evidence remains useful when a sibling fails. Work that cannot possibly contribute after a mandatory predecessor failure should not consume expensive downstream resources.

Condition	Scheduler behavior
Mandatory setup/dependency failure	Mark descendants CASCADE_BLOCKED; do not start them. Independent nodes may continue.
One browser suite PRODUCT/TEST failure	Persist its failed receipt and artifacts; independent browser/build nodes may continue only when their evidence remains useful to diagnosis/next plan.
Source guard stale	Cancel before browser/provider/database acquisition; replan outside scarce lane.
Infrastructure/provider outage	Preserve unaffected completed evidence; reschedule affected infrastructure obligations according to policy.
Finalizer impossible due to known missing mandatory receipt	Stop new work that cannot repair the missing obligation; preserve completed receipts for carry-forward if valid after repair.

13. Carry-Forward Integration with the Runtime

Part I decides whether evidence is preserved. Part II SHALL operationalize that decision by scheduling only INVALIDATED/REBOUND obligations and by making preserved receipts available to the finalizer without pretending they re-executed.

- Before scheduling a reconciled candidate, load prior valid evidence and compute the Part I invalidation manifest.
- Remove PRESERVED obligations from fresh execution queue while creating derived carry-forward receipts.
- Schedule REBOUND steps that regenerate source-bound records without replaying unaffected product proof.
- Schedule INVALIDATED obligations according to typed resources and dependencies.
- Finalizer consumes fresh + preserved + rebound evidence and rejects any obligation without a valid producer.

14. Reference Capacity Profiles

Capacity profiles are machine-readable and environment-qualified. They are not test semantics. The hosted reference profile should be tuned to minimize critical path without creating excessive hosted jobs.

Profile	Minimum/behavior
REFERENCE_CI_MINIMUM	At least 3 node bundle workers, 2 isolated Chromium browser lanes, 1 WebKit lane when required, 1 build lane, run-owned DB clones; external provider capacities declared separately.
DEVELOPER_NORMAL	Capacity determined from current CPU/memory/disk; preserve interactive reserve; normally 1-2 browser lanes if safe.
DEVELOPER_CONTENTENDED	No new timing-sensitive expensive work unless explicitly requested; route to reference CI where possible.
PROVIDER_LIMITED	Concurrency bounded by external service rate/namespace limits; unrelated

Profile	Minimum/behavior
	local lanes continue.

Hosted provider reality

GitHub runner scarcity may still create queue time. Sounding Line must report it honestly and minimize the number of physical jobs exposed to that queue. External scarcity is not evidence that the product is slow and not a reason to lower proof depth.

15. Migration from the Current Suite-Per-Worker Runtime

Version 1.2 may be implemented incrementally. Each step must preserve the existing required check and evidence semantics.

Stage	Change	Safety boundary
Stage A - Telemetry + resource-aware prep	Add detailed timing; stop DB/browser setup for nodes that do not declare those resources; install only declared browser engines.	Same suite-per-worker topology allowed temporarily.
Stage B - Prepared artifacts	Introduce dependency/browser caches and certified SQLite baseline with exact identity verification.	Workers may still be one-suite while reuse correctness is proven.
Stage C - Bundled workers	Group compatible node suites; emit independent receipts; reduce hosted job count.	Runtime-conformance tests prove no test loss/receipt collapse.
Stage D - Typed capacities / dual browser lanes	Replace broad exclusive matrix with resource-specific lanes and dynamic ports.	Queue broker owns actual shared resources only.
Stage E - Preservation-aware scheduling	Execute only INVALIDATED/REBOUND obligations after Part I manifest; finalizer composes carried evidence.	Conservative fallback retained for unsupported cases, but recorded as debt.

16. Part II v1.2 Acceptance Matrix

Scenario	Acceptance requirement
Pure unit/static suite	No DB migration/seed, browser download, or server startup occurs; receipt still passes.
Chromium-only browser suite	Chromium restored; WebKit not restored; exact browser identity recorded.
Certified baseline cache hit	Baseline digest verified; run-owned clone created; baseline remains unchanged after run.
Certified baseline cache miss	Baseline built once from migrations/seed, validated, hashed, then reused by multiple run-owned clones.
Bundle of 4 unit suites	One physical worker emits 4 valid per-suite receipts with exact case reconciliation and independent result/cleanup.
Two independent browser suites	Two isolated lanes run concurrently with different dynamic ports/DB clones/evidence roots and no cross-observation.
Build + browser independent	Build lane overlaps browser lane when no real shared resource exists.
Shared external provider	Provider capacity serializes only provider-dependent nodes; unrelated node/browser lanes continue.
Stale queued candidate	Source guard cancels before browser/provider acquisition and emits

Scenario	Acceptance requirement
	CANCELLED_STALE evidence.
Mandatory browser failure	Downstream useless work is cancelled/cascade-blocked; independent completed receipts persist.
Mainline reconciliation with preserved evidence	Only INVALIDATED/REBOUND nodes are freshly scheduled; finalizer accepts valid carry-forward receipts.
Telemetry completeness	Queue, setup, suite execution, teardown, upload, and finalization timings are present and internally consistent.

Final Part II v1.2 rule

Prepare only what the proof needs. Reuse immutable setup when identity is exact. Clone everything mutable. Schedule by real resources and dependencies, not by broad labels. Keep logical suite evidence exact while using as few physical workers and serial bottlenecks as correctness allows.

R. References and Amendment Closure

- Project Sounding Line Part II - Execution Infrastructure and Parallel Runtime, Version 1.0, July 28, 2026.
- Project Sounding Line Part II - Version 1.1 Amendment Edition, August 10, 2026.
- Companion Sounding Line Parts I and III, Version 1.0 and Version 1.1 amendments.
- Current repository implementation at drafting time: ``.github/workflows/sounding-line-authoritative.yml``, ``.github/workflows/sounding-line-governed-worker.yml``, ``.scripts/sounding-line/planner.mjs``, ``.scripts/sounding-line/adapters.mjs``, ``.scripts/sounding-line/authority.mjs``, ``.policy/registry/finalizer`` files.
- August 2026 hosted authoritative-run evidence demonstrating repeated dependency installation, repeated migration/seed preparation, broad exclusive-wave serialization, browser-engine provisioning duplication, and late browser failure discovery.

All unaffected Version 1.0 and Version 1.1 runtime rules remain governing. Version 1.2 supersedes any implementation interpretation that treats a generic worker template, a fixed port, a broad “exclusive” bucket, or suite-per-VM layout as justification for unnecessary serialization or setup.